

¥

ステップ3: タスクの完了状態を保存・復元する

前のステップでは、タスクの文字列をlocalStorageに保存し、ページを再読み込みした際に復元できるようになりました。このステップでは、各タスクの「完了状態」（チェックボックスの状態）も保存し、復元できるように機能を拡張します。

学習目標

- タスクの状態（未完了・完了）をデータとして管理する方法を理解する。
- チェックボックスの変更イベントを検知し、対応するタスクの状態を更新する方法を学ぶ。
- 更新されたタスク情報をlocalStorageに保存し、ページ読み込み時に復元する方法を習得する。

1. タスクデータに完了状態を追加する

まず、`tasks` 配列に保存する各タスクのデータ構造を変更します。これまではタスクの文字列だけを保存していましたが、各タスクをオブジェクトとして扱い、その中にタスク名（`text`）と完了状態（`completed`）を保持するようにします。

変更前（`script.js` の `tasks` 配列のイメージ）：

```
// tasks = ["牛乳を買う", "宿題をする"];
```

変更後（`script.js` の `tasks` 配列のイメージ）：

```
/*
tasks = [
  { text: "牛乳を買う", completed: false },
  { text: "宿題をする", completed: true }
];
*/
```

`completed` プロパティは、タスクが完了していれば `true`、未完了なら `false` を持つようにします。

2. renderTasks 関数の変更

タスクのデータ構造が変わったため、`renderTasks` 関数も変更が必要です。

- タスクのテキストを表示する際には、`task.text` を参照します。
- 各タスクのチェックボックスの状態を、`task.completed` の値に基づいて設定します。
- チェックボックスがクリックされたときに、対応するタスクの `completed` プロパティを更新し、`saveTasks` 関数を呼び出すイベントリスナーを追加します。

```
// script.js

// ... (既存のコード) ...

function renderTasks() {
  taskList.innerHTML = ''; // リストをクリア
  tasks.forEach((task, index) => {
    const listItem = document.createElement('li');

    // チェックボックスを作成・設定
    const checkbox = document.createElement('input');
    checkbox.type = 'checkbox';
    checkbox.checked = task.completed; // ★変更点: 完了状態を反映
    checkbox.addEventListener('change', () => {
      toggleTaskCompletion(index); // ★変更点: 完了状態を切り替える関数を呼ぶ
    });

    // タスク名を表示するspan要素
    const taskText = document.createElement('span');
    taskText.textContent = task.text; // ★変更点: taskオブジェクトのtextプロパティを参照
    if (task.completed) {
      taskText.classList.add('completed'); // 完了済みのタスクには打ち消し線などを適用
    }

    // 削除ボタン (変更なし)
    const deleteButton = document.createElement('button');
    deleteButton.textContent = '削除';
    deleteButton.addEventListener('click', () => {
      deleteTask(index);
    });

    listItem.appendChild(checkbox);
    listItem.appendChild(taskText);
    listItem.appendChild(deleteButton);
    taskList.appendChild(listItem);
  });
}
```

```
// ★追加: タスクの完了状態を切り替える関数
function toggleTaskCompletion(index) {
  tasks[index].completed = !tasks[index].completed; // 状態を反転
  saveTasks(); // 変更をlocalStorageに保存
  renderTasks(); // 表示を更新
}
```

2.1. データフローと処理の図解（タスク完了状態変更時）

チェックボックスがクリックされてからタスクの状態が保存・再描画されるまでの流れは以下のようになります。

```
``mermaid
graph TD
  A[ユーザーがチェックボックスをクリック] --> B[イベントリスナー発火];
  B --> C[toggleTaskCompletion関数実行 (該当タスクのindexと共に)];
  C --> D[tasks配列内の該当タスクのcompletedプロパティを反転];
  D --> E[saveTasks関数実行];
  E --> F[更新されたtasks配列をJSON文字列に変換];
  F --> G[localStorageに保存 (キー: todoTasks)];
  D --> H[renderTasks関数実行];
  H --> I[画面のリスト全体を再描画 (打ち消し線などスタイルも更新)];
```

CSSで打ち消し線を追加しておく、と視覚的に分かりやすくなります。

```
/* style.css に追加 */
.completed {
  text-decoration: line-through;
  color: #aaa;
}
```

3. addTask 関数の変更

新しいタスクを追加する際も、オブジェクト形式で `tasks` 配列に追加するように変更します。新しいタスクはデフォルトで未完了 (`completed: false`) とします。

```
// script.js

// ... (既存のコード) ...

function addTask() {
  const taskText = taskInput.value.trim();
  if (taskText === '') {
    alert('タスクを入力してください。');
    return;
  }
  // ★変更点: オブジェクトとしてタスクを追加
```

```
tasks.push({ text: taskText, completed: false });
saveTasks();
renderTasks();
taskInput.value = ''; // 入力欄をクリア
}

// ... (既存のコード) ...
```

4. saveTasks 関数の変更

saveTasks 関数は、**tasks** 配列（オブジェクトの配列）をそのまま **JSON.stringify** で文字列に変換して保存するため、大きな変更は必要ありません。

```
// script.js

// ... (既存のコード) ...

const STORAGE_KEY = 'todoTasks';

function saveTasks() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
  // console.log('タスクを保存しました:', tasks); // 確認用ログ
}

// ... (既存のコード) ...
```

5. loadTasks 関数の変更

loadTasks 関数も、localStorageから取得したJSON文字列を **JSON.parse** でオブジェクトの配列に戻すため、大きな変更は必要ありません。ただし、初めてサイトを開いた場合やlocalStorageにデータがない場合を考慮し、**tasks** 配列を空の配列で初期化する処理はそのまま残します。

また、ステップ2で追加した **JSON.parse** のエラーハンドリングは、ここでも有効です。

```
// script.js

let tasks = []; // tasksの初期化は空の配列で良い

// ... (STORAGE_KEY, addTaskButton, taskInput, taskList の定義) ...

function loadTasks() {
  const storedTasks = localStorage.getItem(STORAGE_KEY);
```

```
if (storedTasks) {  
  try {  
    tasks = JSON.parse(storedTasks);  
  } catch (e) {  
    console.error("localStorageからタスクを読み込めませんでした。データが破損している可能性があります。", e);  
    tasks = []; // エラー時は空のタスクリストとして扱う  
  }  
}  
renderTasks(); // 読み込んだタスク（または空のリスト）を表示  
}  
  
// ... (renderTasks, toggleTaskCompletion, addTask, saveTasks, deleteTask の定義) ...  
  
// ページ読み込み時にタスクを読み込む  
loadTasks();
```

6. まとめと動作確認

このステップでは、タスクの完了状態も保存・復元できるように、以下の変更を行いました。

- データ構造の変更:** `tasks` 配列の要素を文字列から `{ text: "タスク名", completed: false }` のようなオブジェクトに変更しました。
- renderTasks 関数の更新:**
 - タスクオブジェクトの `text` プロパティを表示するように変更。
 - タスクオブジェクトの `completed` プロパティに基づいてチェックボックスの状態を設定。
 - チェックボックス変更時に `toggleTaskCompletion` 関数を呼び出すように変更。
 - 完了済タスクにスタイルを適用する処理を追加。
- toggleTaskCompletion 関数の追加:** タスクの完了状態を切り替え、保存し、再描画する関数を追加しました。
- addTask 関数の更新:** 新しいタスクをオブジェクト形式で追加するように変更しました。
- saveTasks, loadTasks 関数の確認:** これらの関数は、オブジェクトの配列を扱う場合でも、`JSON.stringify` と `JSON.parse` によって正しく動作することを確認しました。

動作確認:

- `index.html` をブラウザで開きます。
- いくつかのタスクを追加します。
- いくつかのタスクのチェックボックスをオンにします。
- ページをリロードします。
 - 期待される動作:** タスクのテキストと完了状態（チェックボックスの状態、打ち消し線）がリロード前と同じように復元されていること。

5. 開発者ツールで `localStorage` の `todoTasks` の値を確認し、`completed` プロパティが正しく保存されていることを確認します。

```
// 例: [{"text": "タスク1", "completed": true}, {"text": "タスク2", "completed": false}]
```

7. Q&A

Q1: `toggleTaskCompletion` で `saveTasks()` と `renderTasks()` の両方を呼ぶのはなぜですか？

A1: `saveTasks()` は変更された `tasks` 配列の状態を `localStorage` に永続化するために必要です。一方、`renderTasks()` は画面の表示を最新の `tasks` 配列の状態に合わせて更新するために必要です。例えば、チェックボックスの状態や打ち消し線の表示を正しく反映させるためです。この2つは目的が異なるため、両方呼び出しています。

Q2: タスクオブジェクトに `id` はもう不要なのですか？

A2: このステップ3の時点では、タスクの識別は配列のインデックス (`index`) で行っているため、必ずしも `id` プロパティは必須ではありません。しかし、ステップ1やステップ2の解説では `id: Date.now()` を持つオブジェクトとしてタスクを扱っていました。今後の発展機能（例えばタスクの編集など）で特定のタスクを一意に識別する必要がある性を考えると、`id` を持たせておくことは良い習慣です。この演習では、ステップが進むにつれて必要なプロパティに焦点を当てるため、説明を簡略化している場合があります。最終的な解答例では、`id` も含めた形でデータ管理をすることが推奨されます。

Q3: CSSの `.completed` クラスはどこで定義すれば良いですか？

A3: `style.css` ファイルに以下のように追加します。

```
.completed {
  text-decoration: line-through;
  color: #aaa;
}
```

これにより、`taskText.classList.add('completed');` が実行された際に、該当のスタイルが適用されます。

8. コラム：エラーハンドリングについて（再掲）

実際のアプリケーション開発では、localStorageへの保存・読み込み時にエラーが発生する可能性も考慮する必要があります。

- **localStorageがいっぱいの場合:** `localStorage.setItem()` は `QuotaExceededError` というエラーを投げる可能性があります。
- **不正なJSONデータ:** `localStorage` に手動で不正なJSON文字列が保存されていた場合、`JSON.parse()` がエラーを投げる可能性があります。

演習では簡略化のためエラー処理を省略していますが、より堅牢なアプリケーションを作るためには、`try...catch` 文を使ってこれらのエラーを捕捉し、ユーザーに通知したり、適切に処理したりすることが推奨されます。

例: `loadTasks` での `JSON.parse` のエラーハンドリング

```
function loadTasks() {  
  const storedTasks = localStorage.getItem(STORAGE_KEY);  
  if (storedTasks) {  
    try {  
      tasks = JSON.parse(storedTasks);  
    } catch (e) {  
      console.error("localStorageからタスクを読み込めませんでした。データが破損している可能性があります。", e);  
      tasks = []; // エラー時は空のタスクリストとして扱う  
    }  
  }  
  renderTasks();  
}
```

コラム: localStorageのキーについて

`localStorage.setItem(key, value)` や `localStorage.getItem(key)` で使用する `key` (今回の演習では `STORAGE_KEY` 定数の `'todoTasks'`) は非常に重要です。

- **キー名を間違えると...**
 - 保存時: 意図しないキー名でデータが保存されてしまいます。
 - 読み込み時: 正しいキー名でデータを取得できず、結果としてデータが存在しないかのように振る舞います。
- **確認方法:**
 - ブラウザの開発者ツール (多くのブラウザでF12キーで開けます) の「アプリケーション」タブ (または「ストレージ」タブ) で、localStorageに保存されているキーと値を確認できます。キー名が正しいか、値が期待通りかを確認しましょう。

次のステップでは、タスクを削除した際に、localStorageからも正しく削除されるようにします。